

Capstone Project

Forecasting Household Electricity Consumption with Recurrent Neural Networks

Abstract

This project builds a short-horizon electricity demand forecaster: given the previous 24 hours of household power usage, the model predicts the global active power for the next hour. Two sequential deep learning architectures were developed and compared — a Simple Recurrent Neural Network (RNN) and a Long Short-Term Memory (LSTM) network — to evaluate how well each captures temporal dependencies in real-world energy consumption data. Both models were trained on the same preprocessed dataset and benchmarked on held-out test data using standard regression error metrics.

The Dataset

The project uses the **Individual Household Electric Power Consumption** dataset, which records minute-level power readings from a single household. The raw file contained roughly 2.07 million rows, with features including voltage, global active power, global reactive power, and sub-metering readings for different parts of the household.

Data Preparation

The raw dataset required substantial cleaning and restructuring before it could be used to train a neural network:

- Missing values (marked as '?' in the source file) were filled using forward-fill (ffill), preserving the continuity of the time series rather than dropping rows.
- The dataset was checked for duplicate rows prior to transformation.
- The separate Date and Time columns were merged into a single Datetime field and set as the index.
- Minute-level readings were resampled into hourly averages, compressing the data down to **34,589** hourly records and substantially reducing short-term noise.

Feature Selection and Scaling

Global_active_power was selected as the sole target feature for forecasting. Since recurrent networks train more stably and converge faster on normalized inputs, the target was scaled into the [0, 1] range using a MinMaxScaler prior to sequence generation.

Sequence Construction and Train/Test Split

A sliding-window approach was used to convert the scaled hourly series into supervised learning examples: each input sequence spans **T = 24 hours**, and the model is trained to predict the consumption value at the 25th hour. This produced 34,565 sequence-target pairs, which were then split chronologically (not shuffled, to respect the time-ordering of the data) into:

- **Training set:** 27,652 sequences (80%)
- **Test set:** 6,913 sequences (20%)

Model Architectures

Two comparable architectures were built so that any performance difference could be attributed to the recurrent cell type rather than to differences in model capacity or training setup. Both models share an identical shape: one recurrent layer of 50 units, a dropout layer (rate 0.2) for regularization, a 25-unit ReLU dense layer, and a final single-unit output layer.

Architecture	Recurrent Layer	Trainable Parameters
Simple RNN	SimpleRNN(50, activation='tanh')	3,901
LSTM	LSTM(50, activation='tanh')	11,701

The LSTM's larger parameter count comes from its gating mechanism (input, forget, and output gates), which gives it roughly three times the capacity of the SimpleRNN cell to selectively retain or discard information across the 24-hour window.

Training Configuration

- **Optimizer:** Adam, learning rate = 0.001
- **Loss function:** Mean Squared Error (MSE), monitored alongside Mean Absolute Error (MAE)
- **Early Stopping:** monitored validation loss with a patience of 5 epochs, restoring the best weights to prevent overfitting
- **Epochs / batch size:** up to 30 epochs, batch size 32, with a 10% validation split carved from the training set (sequence order preserved, not shuffled)

Data Visualization

- An initial line plot of hourly Global Active Power over the full date range was produced to inspect overall trends and seasonality before modeling.
- After training, a comparative line chart overlaying actual versus predicted consumption for the first 300 test samples was generated for both the RNN and LSTM models, giving a visual read on how closely each model tracks real usage spikes and dips.

Results

Both models were evaluated on the unseen test set after inverse-transforming predictions back to the original kilowatt scale, so the reported errors are directly interpretable in real electricity units.

Model Architecture	MAE (kW)	RMSE (kW)
Simple RNN	0.3739	0.5102

LSTM	0.3565	0.5068
------	--------	--------

The LSTM model outperforms the Simple RNN on both error metrics, achieving roughly a 4.7% lower MAE and a modest RMSE improvement. This is consistent with expectations: the LSTM's gating mechanism helps it retain relevant information over the full 24-hour look-back window and avoid the vanishing-gradient issues that limit a plain RNN's ability to model longer sequences. That said, the margin between the two models is relatively small, suggesting that at this sequence length (24 hours) the SimpleRNN is still a reasonably competitive, cheaper baseline.

Deployment Readiness

To move the project beyond a notebook experiment, the trained LSTM model was serialized to disk (.keras format) along with the fitted MinMaxScaler (pickled), and Streamlit was installed as the target framework for a lightweight interactive forecasting demo. This packages the model and its preprocessing step together, so predictions on new data can be reproduced consistently outside the training environment.

Conclusion

This project demonstrates an end-to-end time-series forecasting pipeline: cleaning and resampling raw sensor data, framing it as a supervised sequence-prediction problem, and training and comparing two recurrent architectures under identical conditions. The results confirm that LSTM's memory-gating mechanism gives it an edge over a Simple RNN for this 24-hour household electricity forecasting task, while also showing that the improvement, though consistent, is incremental rather than dramatic for a forecast horizon this short.